

SCADE Model Parser

User Guide

Version 1.0 | May 2026

Audience: Engineers working with SCADE Suite model files

Scope: Installation, CLI usage, output interpretation, troubleshooting

1 Introduction

The SCADE Model Parser is a command-line tool that reads SCADE Suite model files (.scade / .xscade) and produces a formatted Microsoft Excel workbook listing all input signals, output signals, and calibration parameters found in the model.

Typical use cases include:

- Generating a signal inventory for systems-level review or ICD (Interface Control Document) preparation.
- Verifying that all calibration parameters (sensors, constants) are accounted for in the calibration plan.
- Producing a diff-friendly artefact between model versions by comparing output Excel files.
- Automating signal extraction as part of a CI/CD pipeline.

1.1 What the Tool Does Not Do

The parser is a read-only extraction tool. It does not:

- Modify or write back to SCADE model files.
- Validate the SCADE model for correctness or completeness.
- Replace SCADE Suite's built-in reporting features.
- Parse .etp (SCADE project environment) files — only .scade / .xscade model files.

2 Prerequisites and Installation

2.1 System Requirements

Requirement	Minimum	Notes
Python	3.10	Download from python.org. Verify with: <code>python --version</code>
pip	21.0+	Bundled with Python 3.10+.
openpyxl	3.1.0	Required. Installed via pip (see below).
defusedxml	0.7.1	Strongly recommended for security. Installed via pip.
Disk space	< 5 MB	For the parser script and its dependencies.
OS	Windows / macOS / Linux	No platform-specific code.

2.2 Installation Steps

1. Copy `scade_parser.py` to a folder of your choice (e.g. `C:\Tools\scade_parser\`).
2. Open a terminal (Command Prompt, PowerShell, or bash).
3. Install the required Python packages:

```
pip install openpyxl defusedxml
```

4. Verify the installation:

```
python scade_parser.py --help
```

Expected output:

```
usage: scade_parser [-h] [--output OUTPUT] INPUT

Parse SCADE model files (.scade / .xscade) and extract
input signals, output signals, and calibration parameters
into a formatted Excel workbook.
```

TIP: Add the folder containing `scade_parser.py` to your system PATH, or create an alias, so you can run it from any directory.

2.3 Security Behaviour Without defusedxml

`defusedxml` is listed as 'strongly recommended' rather than strictly required because the parser includes a `stdlib` fallback. However, the two modes are NOT security-equivalent:

Scenario	With defusedxml	Without defusedxml (stdlib fallback)
XXE entity injection	Blocked — <code>EntitiesForbidden</code> raised immediately	Partially blocked — <code>DOCTYPE</code> and external entity resolver rejected, but protection depth is lower
Billion-laugh XML bomb	Blocked — <code>defusedxml</code>	Partially mitigated —

	detects recursive entity expansion	DOCTYPE rejected; deeply nested but non-entity attacks may still consume memory
Standard well-formed XML	Parsed normally	Parsed normally
Malformed / truncated XML	ParseError raised; file skipped	ParseError raised; file skipped

IMPORTANT: Always install defusedxml in CI/CD pipelines, shared build environments, and any workflow where .scade files may originate from outside your team. The stdlib fallback is acceptable only for local development on files you control.

3 Quick Start

3.1 Parse a Single File

```
python scade_parser.py my_model.scade
```

This produces a file named `scade_signals_YYYYMMDD_HHMMSS.xlsx` in the current directory.

3.2 Parse a Directory of Models

```
python scade_parser.py ./models/
```

The parser recursively scans the `models/` directory for all `.scade` and `.xscade` files and combines their results into a single Excel workbook.

3.3 Specify an Output File

```
python scade_parser.py ./models/ --output VehicleControl_Signals.xlsx
```

3.4 Sample Console Output

```
SCADE Model Parser
=====
Input  : /home/user/models/
Output : /home/user/VehicleControl_Signals.xlsx

Parsing : BrakeController.scade
Parsing : ThrottleController.scade
Parsing : Diagnostics.scade

Results
-----
Input signals      : 12
Output signals     : 8
Calibration parameters : 15

Writing Excel workbook ...
Sheet 'Input Signals': 12 rows
Sheet 'Output Signals': 8 rows
Sheet 'Calibration Parameters': 15 rows

✓ Saved: /home/user/VehicleControl_Signals.xlsx
```

4 CLI Reference

4.1 Synopsis

```
python scade_parser.py INPUT [--output OUTPUT]
```

4.2 Arguments

Argument	Required	Default	Description
INPUT	Yes	—	Path to a .scade file or a directory containing .scade files.
--output / -o	No	scade_signals_YYYYMMDD_HHMMSS.xlsx	Output Excel file path. Must end in .xlsx.
--help / -h	No	—	Display help and exit.

4.3 Exit Codes

Code	Meaning	When It Occurs
0	Success	Parsing completed and Excel file written.
0	No data (warning)	Parsing completed but no signals were found. A [WARNING] is printed to stderr. No Excel file is written. This also occurs when every file in a directory was skipped due to per-file errors — the individual skip messages appear on stderr but the overall exit code is 0. Check stderr output to distinguish this from a genuine empty model.
1	Error	Invalid arguments, non-existent or non-.xlsx output path, no write permission for the output directory, or Excel write failure (e.g. file open in Excel).

4.4 Usage Examples

Parse a single file, default output name:

```
python scade_parser.py engine_control.scade
```

Parse a directory, custom output name:

```
python scade_parser.py ./scade_models/ -o ECU_Signals_v2.xlsx
```

Use an absolute path:

```
python scade_parser.py C:\Projects\ECU\models\ --output C:\
Reports\signals.xlsx
```

In a shell script (check exit code):

```
python scade_parser.py ./models/ --output signals.xlsx && echo 'OK' || echo
'FAILED'
```

5 Understanding the Output

The Excel workbook produced by the parser contains four sheets. Each sheet is accessible via the tabs at the bottom of the Excel window.

5.1 Summary Sheet

The first sheet provides a quick overview: the date and time the file was generated, the output filename, and the total count of records in each category. Use this sheet to confirm at a glance that the expected number of signals was extracted.

5.2 Input Signals Sheet

Column	Example	Description
Name	BrakePedalPos	The signal variable name as declared in the SCADE model.
Type	float32	The SCADE data type (e.g. float32, bool, int32, uint8).
Operator	BrakeController	The SCADE operator that owns this signal.
Package	VehicleControl	The package (or nested package path) containing the operator.
Comment	Pedal position 0-100%	Optional description from the SCADE model.
Source File	brake_ctrl.scade	The .scade file from which this signal was extracted.

5.3 Output Signals Sheet

Contains the same columns as the Input Signals sheet, but lists all output signals.

5.4 Calibration Parameters Sheet

Column	Example	Description
Name	CAL_MaxBrakePressure	The calibration parameter name.
Type	float32	The SCADE data type.
Kind	Sensor	'Sensor' or 'Constant' — the SCADE element type.
Default Value	250.0	Initial or default value (empty if not specified in the model).
Operator		Enclosing operator name (often empty for top-level parameters).
Package	VehicleControl	Package path.
Comment	Max brake pressure [bar]	Optional description.
Source File	brake_ctrl.scade	Source file.

5.5 Excel Features

- Frozen header row — the column headers stay visible while you scroll down.
- Auto-filter — click the drop-down arrows in the header row to filter by any column.
- Alternating row colours — alternate rows are shaded light blue for readability.

- Auto-sized columns — column widths adjust to content (maximum 55 characters).

6 Supported SCADE XML Patterns

Different versions of SCADE Suite and different modelling conventions produce slightly different XML structures. The parser handles all three common patterns for input/output declarations and two patterns for calibration parameters.

6.1 Input and Output Patterns

Pattern	XML Structure	Supported?
Flat Attribute	<code><Input Name="Speed" TypeRef="float32"/></code>	Yes
Variable Children	<code><Input><Variable Name="Speed" TypeRef="float32"/></Input></code>	Yes
Direction Attribute	<code><Variable Name="Speed" TypeRef="float32" Direction="in"/></code>	Yes

6.2 Calibration Parameter Patterns

Element	Example	Supported?
Sensor (attribute default)	<code><Sensor Name="K_MAX" TypeRef="float32" Default="100.0"/></code>	Yes
Sensor (child element)	<code><Sensor Name="K_MAX" TypeRef="float32"><Default>100.0</Default></Sensor></code>	Yes
Const (attribute value)	<code><Const Name="PI" TypeRef="float64" Value="3.14159"/></code>	Yes
Constant (child element)	<code><Constant Name="PI" TypeRef="float64"><Value>3.14159</Value></Constant></code>	Yes

NOTE: If your SCADE model uses element names different from those listed above, the signals will not be extracted. Refer to the Architecture Document section 5 for instructions on extending the parser to support additional element types.

7 Working with Directories

When a directory is passed as the input, the parser recursively discovers all .scade and .xscade files within that directory and any subdirectories. All results are combined into a single Excel workbook. The Source File column in each sheet indicates which file each record came from.

7.1 Example Directory Structure

```
models/
├── VehicleControl/
│   ├── BrakeController.scade
│   └── ThrottleController.scade
├── Diagnostics/
│   └── FaultMonitor.scade
└── Common/
    └── Types.xscade
```

```
python scade_parser.py ./models/ --output full_model_signals.xlsx
```

This will extract signals from all four files and produce a single workbook. Files are processed in alphabetical order by filename.

7.2 Filtering Results in Excel

To view signals from a specific file after parsing a directory, use Excel's auto-filter on the Source File column:

5. Open the workbook in Excel.
6. Click the drop-down arrow in the Source File column header.
7. Select only the file name(s) you are interested in.
8. The sheet will display only rows from those files.

8 Troubleshooting

Symptom	Cause	Resolution
No data extracted — [WARNING] message appears	The XML element names in the model file do not match the expected names (Input, Output, Sensor, Const, Constant).	(1) Open the .scade file in a text editor and check the XML structure. (2) Verify that input/output interfaces use <Input>, <Output>, or <Variable Direction='in'/'out'> elements. (3) If different element names are used, refer to the Architecture Document (section 5.1) to extend the parser.
XML parse error: EntitiesForbidden	The .scade file contains a DOCTYPE declaration or entity reference. This is a security block, not a bug.	(1) If the file is from a trusted source and you are confident it is safe, inspect the DOCTYPE block manually and remove it before parsing. (2) If the file was received from an untrusted source, do not bypass this protection.
File skipped: Unsupported extension	A file with an extension other than .scade or .xscade was found in the directory.	(1) This is normal — the parser ignores non-model files. (2) If a valid model file has an unexpected extension, rename it to .scade before running the parser.
Output path problem: Output directory does not exist	The directory specified in --output does not exist.	(1) Create the directory manually before running the parser. (2) Or specify an output path in an existing directory.
PermissionError: Cannot write to output file	The output Excel file is currently open in Microsoft Excel.	(1) Close the file in Excel, then re-run the parser. (2) Or specify a different output filename with --output.
File too large: exceeds 50 MB limit	A .scade file is larger than the 50 MB safety limit.	(1) Verify that the file is a legitimate SCAD model file and not corrupted. (2) If legitimate, contact the tool maintainer to increase MAX_FILE_SIZE_MB in scade_parser.py (line near the top of the file).
Type shown as 'unknown'	The parser could not find a recognised type attribute on the element.	(1) Open the .scade file and check what attribute name is used for the data type. (2) Common alternatives: TypeRef, typeRef, Type, DataType. (3) If a new attribute name is found, it can be added to the _attr() call in _collect_var() — see Architecture Document section 5.4.

Exit code 0 but no Excel file produced; stderr shows [SKIP] messages	Every file in the directory was skipped due to per-file errors (e.g. all files exceeded the 50 MB limit, had parse errors, or had unsupported extensions). The tool exits 0 because no unrecoverable top-level error occurred.	(1) Read all [SKIP] lines on stderr — each line explains why that specific file was skipped. (2) In CI pipelines, redirect stderr to a log file and check it even when the exit code is 0: <code>python scade_parser.py ./models/ 2>parse_errors.log</code> (3) If you need the pipeline to fail when no output is produced, check for the absence of the output .xlsx file as an additional CI step.
---	---	---

9 Frequently Asked Questions

Q: Can I run the parser on .etp SCADE project files?

A: No. The parser only reads .scade and .xscade model files. An .etp project file lists the model files in the project — you can point the parser at the directory containing those files instead.

Q: The model has many packages nested inside each other. Will the parser handle this?

A: Yes. The parser traverses the XML tree recursively and tracks the full package path (e.g. System/VehicleControl/Braking) as it descends. All nested packages are handled correctly.

Q: Will the parser work with SCADE Display or SCADE LifeCycle files?

A: The parser is designed specifically for SCADE Suite model files. SCADE Display and SCADE LifeCycle use different XML schemas and the parser may not extract signals from them correctly.

Q: Can I import the parser into my own Python script?

A: Yes. Import `parse_path` and `write_excel` directly: `from scade_parser import parse_path, write_excel`. These functions work independently of the CLI.

Q: The output file already exists. Will the parser overwrite it?

A: Yes. The parser will overwrite an existing file with the same name. If the file is open in Excel, a `PermissionError` will be raised -- close the file in Excel first.

Q: Can the parser extract signals from multiple .scade files into separate sheets?

A: Not in its current form -- all results are combined into three category sheets. To separate results by file, use Excel's auto-filter on the Source File column, or run the parser once per file with a different `--output` for each.

Q: How do I add support for a new SCADE element type?

A: Add an `elif` branch in the `_walk()` method in `scade_parser.py`. See the Architecture Document section 5.1 for a step-by-step guide.

Q: Is there a log file?

A: No. All output goes to the terminal: normal messages to `stdout` and error/warning messages (prefixed with `[ERROR]`, `[SKIP]`, or `[WARNING]`) to `stderr`. Redirect `stderr` to capture errors: `python scade_parser.py ./models/ 2>errors.txt`

Q: The tool exited with code 0 but produced no Excel file. What happened?

A: This means every file was skipped (per-file errors) or no signals were found. Check `stderr` for `[SKIP]` and `[WARNING]` messages. In CI, redirect `stderr` to a log and add a check that the expected .xlsx file exists after the parser runs.

10 Revision History

Version	Date	Author	Summary
1.0	May 2026	Engineering Team	Initial release.